MONTH 1 — PYTHON FUNDAMENTALS + FRONTEND BASICS

DAY 1: Introduction to Python — Variables, Data Types & print()

Week 1 · Python Basics · Beginner → Professional Python Web Developer Track

---

1. Theory

Python is the foundation of this entire roadmap. Before touching Flask, Django, databases, or deployment, you need to be fluent in core Python — because every web framework you'll learn is just Python code, organized with extra rules.

1.1 Why Python First, Before Flask/Django

- Flask and Django are written in Python — their syntax IS Python syntax.
- Reading a stack trace, a Django model, or a Flask route all require solid Python basics.
- Weak fundamentals here cause confusion in every later month — this is the highest-leverage day of the whole roadmap.

1.2 Variables

A variable is a named container that stores a value in memory. Python is dynamically typed — you don't declare a type, Python infers it from the value you assign.

- Variable names are case-sensitive (username and Username are different).
- Must start with a letter or underscore, not a digit.
- Cannot use Python reserved keywords (e.g. class, for, if, import).
- Convention: snake_case for variables (user_name, not UserName) — this matches Flask/Django naming conventions you'll use later.

1.3 Core Data Types

- int — whole numbers, e.g. 25, -3, 1000
- float — decimal numbers, e.g. 3.14, -0.5
- str — text, written in quotes, e.g. "Web Developer"
- bool — True or False
- NoneType — represents "no value", written as None (this is exactly what Django/Flask return when a database record doesn't exist)

1.4 print() and Comments

print() displays output to the console — your main tool for checking what code is doing before you ever see a browser involved. A comment (starting with #) is text Python ignores, used to explain code to humans reading it — including future-you, six weeks into Django.

2. Syntax

Declaring variables and printing values:

```
# Variable assignment — no type keyword needed
username = "sara_dev"      # str
age = 24                    # int
rating = 4.8                # float
is_active = True            # bool
profile_pic = None          # NoneType

# print() displays values
print(username)
print("Age:", age)

# type() reveals the data type
print(type(age))            # <class 'int'>

# f-strings: the modern way to build text with variables
print(f"{username} is {age} years old.")

# input() reads text typed by the user (always returns a string)
role = input("Enter your role: ")
print("Role selected:", role)
```

3. Small Example

A short program that introduces itself using variables and an f-string:

```
track = "Python Web Developer"
day = 1
hours_today = 2

print(f"Welcome to Day {day} of the {track} roadmap!")
print(f"Today's focused practice time: {hours_today} hours.")
print("Goal: get comfortable with variables, data types, and print().")
```

Expected output:

```
Welcome to Day 1 of the Python Web Developer roadmap!
Today's focused practice time: 2 hours.
Goal: get comfortable with variables, data types, and print().
```

4. 2-Hour VS Code Practical

Open VS Code, create a folder called day01, and inside it create two files: program1.py and program2.py. Run each with the ▶️ Run button or python program1.py in the terminal. Aim for roughly 1 hour per program — read the task, write the code yourself before checking the Solutions section.

Program 1 (≈60 min) — Developer Profile Card

Write a program that:

1. Stores your name, age, city, and main tech stack goal (e.g. "Flask & Django") in four variables.
2. Stores a boolean variable is_learning_web set to True.
3. Prints all details in a neatly formatted "profile card" using f-strings, e.g.:

```
===== DEVELOPER PROFILE =====
Name        : Sara
Age         : 24
City        : Karachi
Goal Stack  : Flask & Django
Learning Web: True
==============================
```

Bonus: use type() to print the data type of each variable underneath the card.

Program 2 (≈60 min) — Simple Project Estimator

Write a program that:

4. Uses input() to ask the user for their name and how many web projects they plan to build this year (comes in as a string).
5. Converts the project count to an int using int().
6. Calculates and prints roughly how many days per project they'll have, assuming 112 learning days total (use round() for a clean number).
7. Prints a friendly summary sentence using an f-string.

Example run:

```
Enter your name: Bilal
How many projects do you plan to build this year? 4
Hi Bilal! At this pace, you have about 28 learning days per project.
```

5. Solutions

Solution — Program 1

```
name = "Sara"
age = 24
city = "Karachi"
goal_stack = "Flask & Django"
```

```
is_learning_web = True

print("===== DEVELOPER PROFILE =====")
print(f"Name        : {name}")
print(f"Age         : {age}")
print(f"City        : {city}")
print(f"Goal Stack  : {goal_stack}")
print(f"Learning Web: {is_learning_web}")
print("============================")

# Bonus
print(type(name), type(age), type(city), type(goal_stack),
type(is_learning_web))
```

Solution — Program 2

```
name = input("Enter your name: ")
projects_str = input("How many projects do you plan to build this year? ")
projects = int(projects_str)        # convert text -> whole number

total_days = 112
days_per_project = round(total_days / projects)

print(f"Hi {name}! At this pace, you have about {days_per_project}
learning days per project.")
```

Common beginner mistake: forgetting int(projects_str) and trying total_days / projects_str directly — Python cannot divide a number by a string, so it raises a TypeError.

6. Mini Project — "My Web Dev Journey" Intro Script

Combine everything from today into one small script, web_journey.py:

8. Ask the user for their name and current profession using input().
9. Store today's progress info as variables: course_day = 1, total_days = 112.
10. Calculate progress percentage: (course_day / total_days) * 100, rounded to 2 decimal places using round().
11. Print a motivational summary using an f-string, e.g. "Hi <name>! You're a <profession> starting Day 1 of 112 (0.89% complete). Let's build toward Full-Stack Python Web Developer level!"

This mini project reinforces variables, data types, type conversion, input/output, and f-strings — the exact building blocks every later day (Flask, Django, REST APIs, Deployment) will build on.

Next: Day 2 will build on this with arithmetic operators, string methods, and basic user-input validation.